BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Lê Ngọc Phúc
MSSV / Lớp	3124720053

Quy ước chung: dùng graph search có explored set; kiểm tra đích khi lấy nút ra khỏi frontier (nên nút đích G cũng nằm trong Explored Set); khi hòa thì chọn theo thứ tự alphabet (BFS/DFS: các nút con được thêm/duyệt theo thứ tự alphabet).

Bài 1. Các thuật toán tìm kiếm

Đồ thị: S→A(3), S→C(2), S→D(2), C→F(1), D→B(3), D→G(8), B→E(2), F→E(0), F→G(4), E→G(2).

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B, G
Path	S → D → G
Total Cost	10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S → C → F → E → G
Total Cost	2 + 1 + 0 + 2 = 5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G → tìm thấy đích

Path cuối cùng: S → D → G

Total Cost: 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

BFS dùng frontier dạng hàng đợi FIFO nên duyệt theo từng tầng; vì vậy nó tìm được lời giải có ít cạnh (độ sâu) nhất, và tối ưu khi mọi cạnh có chi phí bằng nhau (cần nhiều bộ nhớ O(b^d)). DFS dùng frontier dạng ngăn xếp LIFO, đi sâu hết một nhánh rồi mới quay lui, nên tốn ít bộ nhớ (O(bm)) nhưng không đảm bảo tối ưu: ở đây DFS trả về đường chi phí 5 (S-C-F-E-G) rẻ hơn BFS (10) chỉ do tình cờ, còn BFS tối ưu theo số bước chứ không theo chi phí. Cả hai đều không đảm bảo tối ưu về chi phí khi cạnh có trọng số khác nhau (cần UCS/A*).

Bài 2. Tháp Hà Nội

2.1. Biểu diễn trạng thái

Trạng thái là bộ n phần tử ($p_1, p_2, \ldots, p_n$) với $p_k \in \{1,2,3\}$ là cọc đang chứa đĩa thứ k (đĩa 1 nhỏ nhất, đĩa n lớn nhất). Vì trên mỗi cọc các đĩa luôn xếp từ lớn đến nhỏ nên thứ tự trên cọc được xác định duy nhất bởi kích thước; biểu diễn này đủ để xác định đĩa nào ở cọc nào và các hành động hợp lệ.

2.2. Kích thước không gian trạng thái

Mỗi đĩa độc lập chọn 1 trong 3 cọc nên có 3^n trạng thái (mọi cách gán đều hợp lệ vì thứ tự xếp được suy ra từ kích thước). Ví dụ n=3 → 27 trạng thái.

2.3. Trạng thái bắt đầu

S0 = (1, 1, …, 1): tất cả n đĩa nằm trên cọc 1 (đĩa lớn ở dưới, nhỏ ở trên).

2.4. Các hành động hợp lệ

Actions(s) = tập các cặp (i → j), i ≠ j. Hành động hợp lệ khi: (1) cọc i không rỗng; (2) đĩa được chuyển là đĩa trên cùng của cọc i, tức đĩa nhỏ nhất k trên cọc i (không có đĩa j<k nào trên cọc i); (3) cọc j rỗng hoặc đĩa trên cùng của cọc j lớn hơn đĩa k (không có đĩa nhỏ hơn k trên cọc j). Kết quả: p_k đổi từ i sang j. Mỗi lần chỉ chuyển một đĩa và không đặt đĩa lớn lên đĩa nhỏ.

2.5. Goal Test

goalTest(state) = True khi và chỉ khi $p_k = 3$ với mọi k = 1..n, tức toàn bộ đĩa đã nằm trên cọc thứ ba (số bước tối thiểu $2^n - 1$).

Bài 3. Thiết kế và phân tích Heuristic

Đồ thị: S→A(5), S→B(3), A→C(4), A→D(9), B→D(1), C→D(2), C→G(6), D→G(4). Chi phí tối ưu thật h*: S=8, A=10, B=5, C=6, D=4, G=0.

3.1. Heuristic admissible và consistent

Node	h(n)
S	6
A	7
B	4
C	5
D	4
G	0

Admissible: h ≤ h* ở mọi nút (6≤8, 7≤10, 4≤5, 5≤6, 4≤4). Consistent: S→A: 6≤5+7; S→B: 6≤3+4; A→C: 7≤4+5; A→D: 7≤9+4; B→D: 4≤1+4; C→D: 5≤2+4; C→G: 5≤6+0; D→G: 4≤4+0 — đều thỏa.

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: S (f=6), B (f=3+4=7), D (f=4+4=8), G (f=8)

Path: S → B → D → G

Total Cost: 3 + 1 + 4 = 8 (tối ưu)

3.3. Heuristic admissible nhưng không consistent

Chọn h: S=5, A=7, B=2, C=6, D=1, G=0. Admissible vì 5≤8, 7≤10, 2≤5, 6≤6, 1≤4. Không consistent vì cạnh C→D: h(C)=6 > c(C,D)+h(D) = 2+1 = 3 (vi phạm h(n) ≤ c(n,n')+h(n')).

3.4. Câu hỏi mở rộng

Giả sử h consistent và h(G)=0. Với mọi nút n và đường đi tối ưu $n=n_0 \to n_1 \to \ldots \to n_k=G$: $h(n_0) \le c(n_0,n_1)+h(n_1) \le c(n_0,n_1)+c(n_1,n_2)+h(n_2) \le \ldots \le$ tổng chi phí đường đi + h(G) = h*(n). Vậy $h(n) \le h^*(n)$, tức admissible (quy nạp theo độ dài đường đi tới đích).

Bài 4. Treasure Hunting (Tìm kho báu)

Đồ thị: B→A(2); A→C(2), A→D(1), A→E(3); C→F(2), C→G(1); D→G(1), D→H(3); E→H(1), E→I(1); F→B(3). Kho báu (đích): E, G. Heuristic: A=2, B=4, C=1, D=1, E=0, F=6, G=0, H=0, I=0. Bắt đầu từ A.

4.1. Depth-First Search

Explored Set: A, C, F, B, G (A→C→F→B; B chỉ đi về A đã duyệt nên quay lui, rồi C→G)

Path returned: A → C → G (cost 3)

4.2. Breadth-First Search

Explored Set: A, C, D, E

Path returned: A → E (cost 3; vào được kho báu E ở độ sâu 1)

4.3. Uniform-Cost Search

Explored Set: A (g=0), D (1), C (2), G (2) — C và G cùng g=2, chọn C trước theo alphabet

Path returned: A → D → G

Cost: 2

4.4. Greedy Search

Explored Set: A, E (h(E)=0 nhỏ nhất)

Path returned: A → E

Cost: 3

4.5. A* Search

Explored Set: A (f=2), D (f=1+1=2), G (f=2+0=2)

Path returned: A → D → G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A → C → G	3
BFS	Độ sâu nhỏ nhất	A → E	3
UCS	g(n)	A → D → G	2
Greedy	h(n)	A → E	3
A*	g(n)+h(n)	A → D → G	2

Bài 5. Câu hỏi Đúng/Sai

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Đúng / Sai: Sai

Giải thích: DFS có thể mở rộng ít nút hơn. Ví dụ: S→A (cost 10, A là đích), S→B (1), B→A (1); h(S)=2, h(B)=1, h(A)=0 (admissible). DFS (duyệt A trước) chỉ mở rộng S, A (2 nút); A* mở rộng S, B (f=2), A (f=2) (3 nút) vì f(A trực tiếp)=10.

Câu 2. Quân xe … Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

Đúng / Sai: Sai

Giải thích: Xe đi được nhiều ô trong một nước nên Manhattan distance đánh giá quá cao. Ví dụ A=(1,1), B=(1,8): xe cần 1 nước nhưng Manhattan = 7 > 1.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: Đúng

Giải thích: Pacman đi theo 4 hướng, mỗi bước cost 1, nên đường thực tế ≥ Manhattan ≥ Euclidean (đường thẳng là ngắn nhất, tường chỉ làm đường dài thêm). Vậy Euclidean không bao giờ vượt chi phí thật.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai

Giải thích: Tổng có thể vượt chi phí thật. Ví dụ $h_1=h_2=h^*$ (đều admissible) thì $h_1+h_2=2h^* > h^*$ khi $h^*>0$.

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai

Giải thích: Chiều ngược lại mới đúng: consistent ⇒ admissible. Heuristic ở 3.3 (S=5, A=7, B=2, C=6, D=1, G=0) admissible nhưng không consistent (cạnh C→D).

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích: Graph search cần heuristic consistent; với admissible nhưng không consistent, nút có thể bị đóng với g chưa tối ưu. Ví dụ: S→A(1), S→B(1), A→C(3), B→C(1), C→G(5); h(S)=h(A)=h(C)=0, h(B)=6 (admissible vì h*(B)=6). A* graph search mở rộng S, A, C (g=4), rồi B; khi tới C lần nữa (g=2) thì bị bỏ qua vì C đã trong explored set → trả về S-A-C-G cost 9, trong khi tối ưu S-B-C-G cost 7.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Họ và tên	Lê Ngọc Phúc
MSSV / Lớp	3124720053

Câu 1. Tìm kiếm và heuristic

Quy ước tọa độ (x, y): x là cột (0→4 từ trái sang phải), y là hàng (0→4 từ dưới lên). Xuất phát (0,0) hướng Bắc, v=0; đích (0,4) hướng Tây, v=0. Mỗi hành động tốn 1 đơn vị chi phí.

a) Chuỗi hành động ít nhất theo nhánh ngoài cùng bên trái (đi qua ô (1,2)):

Vì ô (0,1) bị tường chắn phía trên (0,0) nên phải đi sang Đông trước:

1. Rẽ phải (Bắc→Đông), v=0, ở (0,0)

2. Tăng tốc → v=1, tới (1,0)

3. Giảm tốc → v=0, đứng yên ở (1,0) (phải dừng mới được rẽ)

4. Rẽ trái (Đông→Bắc)

5. Tăng tốc → v=1, tới (1,1)

6. Giữ tốc độ → v=1, tới (1,2)

7. Giữ tốc độ → v=1, tới (1,3)

8. Giảm tốc → v=0, đứng yên ở (1,3) (tường phía trên (1,3) nên không đi tiếp được)

9. Rẽ trái (Bắc→Tây)

10. Tăng tốc → v=1, tới (0,3)

11. Giảm tốc → v=0

12. Rẽ phải (Tây→Bắc)

13. Tăng tốc → v=1, tới (0,4)

14. Giảm tốc → v=0, dừng tại (0,4)

15. Rẽ trái (Bắc→Tây) → đúng hướng đích (Tây), v=0.

Tổng: 15 hành động ⇒ chi phí 15.

b) Kích thước không gian trạng thái (lưới M × N, vận tốc tối đa Vmax):

Một trạng thái là (x, y, hướng, vận tốc): x có M giá trị, y có N giá trị, hướng có 4 giá trị (Bắc, Nam, Đông, Tây), vận tốc có Vmax+1 giá trị (0..Vmax).

⇒ Số trạng thái = M × N × 4 × (Vmax + 1).

c) Có thể tồn tại trạng thái không có nút con không?

☒ Có ☐ Không

Có. Ví dụ trạng thái (1, 3, Bắc, v=2): Tăng tốc (v=3 đi 3 ô), Giữ tốc độ (đi 2 ô) và Giảm tốc (v=1 đi 1 ô) đều đâm vào tường phía trên (1,3); không được rẽ vì $v \neq 0$. Vậy trạng thái này không có hành động hợp lệ nào (ngõ cụt).

d) Hệ số phân nhánh lớn nhất:

Tối đa 3 hành động hợp lệ ở mỗi trạng thái:

• Khi v = 0: Tăng tốc, Rẽ trái, Rẽ phải (Giữ tốc độ và Giảm tốc không hợp lệ).

• Khi $0 < v < Vmax$: Tăng tốc, Giữ tốc độ, Giảm tốc (không được rẽ).

• Khi v = Vmax: chỉ còn Giữ tốc độ, Giảm tốc (2 hành động).

⇒ b = 3. Ví dụ trạng thái (1, 1, Bắc, v=0): Tăng tốc (tới (1,2)), Rẽ trái, Rẽ phải.

e) Khoảng cách Manhattan tới cửa ra có là heuristic chấp nhận được không?

☐ Có ☒ Không

Không. Một hành động có thể đưa xe đi nhiều ô (tới Vmax ô) nên Manhattan có thể lớn hơn chi phí thật. Ví dụ Vmax=3, hành lang thẳng dài 9 ô, xe đang quay đúng hướng đích với v=0: Manhattan = 9, nhưng chỉ cần 6 hành động: Tăng, Tăng, Tăng (đi 1+2+3=6 ô), Giảm (2 ô), Giảm (1 ô), Giảm (0 ô) → tổng 9 ô, dừng đúng đích. Chi phí thật $6 < h = 9$.

f) Heuristic h = khoảng cách Manhattan / Vmax có admissible không?

☒ Có ☐ Không

Có. Mỗi hành động làm xe tiến tối đa Vmax ô, nên để đi hết khoảng cách Manhattan D cần ít nhất D / Vmax hành động; ngoài ra còn phải dừng (v=0) và rẽ đúng hướng (chỉ làm tăng chi phí thật). Do đó $h \leq$ chi phí thật và h không bao giờ đánh giá quá cao (không có phản ví dụ như câu e).

g) A* Tree Search với heuristic không admissible: tính đầy đủ (completeness) có thể thay đổi không?

☐ Có ☒ Không

Không. Với đồ thị hữu hạn, heuristic không âm (hữu hạn) và chi phí hành động dương, giá trị $f = g + h$ của các nút trên frontier cuối cùng tăng vượt mọi ngưỡng nên mọi nút đều được mở rộng sau hữu hạn bước; vì vậy A* vẫn tìm ra lời giải nếu có. Heuristic không admissible chỉ ảnh hưởng tới tính tối ưu.

h) Tính tối ưu của A* Tree Search với heuristic không admissible có thể thay đổi không?

☒ Có ☐ Không

Có. Heuristic đánh giá quá cao một nút nằm trên đường tối ưu có thể làm f của nó lớn hơn f của một đích kém tối ưu, nên đích kém tối ưu được lấy ra trước và A* trả về lời giải không tối ưu.

i) Lý do hợp lý để dùng heuristic không admissible (chọn tất cả phương án phù hợp):

☒ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☒ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☒ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Phương án 3 sai vì (câu h) tính tối ưu có thể mất. Phương án 4 là cách cắt tỉa: gán h rất lớn (∞) cho các vùng không muốn xét thì A* sẽ không mở rộng chúng (đổi lại có thể mất tối ưu).

Câu 2. Các nút tìm kiếm

Tree search; mỗi nút n có g(n), h(n) admissible và độ sâu d(n). n_c là nút đích có chi phí nhỏ nhất, n_s là nút đích nông nhất.

a) Breadth-First Search (BFS)

Bất đẳng thức: $d(n) \leq d(n_s)$ (mọi nút có độ sâu không vượt quá độ sâu đích nông nhất; nếu phá hòa thuận lợi thì chỉ cần $d(n) < d(n_s)$).

b) Uniform-Cost Search (UCS)

Bất đẳng thức: $g(n) \leq g(n_c)$ (mọi nút có chi phí tích lũy không vượt quá chi phí đích tối ưu).

c) A* Search, h nhất quán

Bất đẳng thức: $g(n) + h(n) \leq g(n_c)$ (tức $f(n) \leq$ chi phí tối ưu, vì $h(n_c)=0$).

Câu 3. Tìm kiếm trên đồ thị

Đồ thị vô hướng: A–C 4, A–B 3, B–C 2, B–E 5, B–D 3, C–E 2, D–E 1, E–F 3, E–G 6, F–G 2. Bắt đầu A, đích G.

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

a) Đường đi mà mỗi thuật toán có thể trả về (xét mọi cách phá hòa):

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	☐	☒	☐	☐
Greedy với h1	☐	☐	☐	☒ A-B-D-E-G hoặc A-B-E-G
Greedy với h2	☒	☐	☐	☐
A* với h1	☐	☐	☒	☐
A* với h2	☐	☒	☐	☐

Giải thích: UCS và A* với h2 trả về đường tối ưu A-C-E-F-G (cost 11). Greedy với h1: từ A chọn B (h=6), từ B hòa giữa D và E (cùng h=3) → A-B-D-E-G (13) hoặc A-B-E-G (14) — cả hai đều thuộc cột "Khác". Greedy với h2: A→C (6)→E (5)→G (0) = A-C-E-G (12). A* với h1 (h1 không admissible) trả về A-B-D-E-F-G (12), không tối ưu.

b) Chi phí của đường đi tối ưu từ A đến G theo UCS:

Đáp án: 11 (đường A-C-E-F-G = 4 + 2 + 3 + 2).

c) Heuristic h1:

Admissible: ☐ Có ☒ Không Consistent: ☐ Có ☒ Không

Chi phí tối ưu thật h*: A=11, B=9, C=7, D=6, E=5, F=2, G=0. h1(A)=12 > h*(A)=11 nên h1 không admissible; do đó cũng không consistent (consistent ⇒ admissible).

d) Heuristic h2:

Admissible: ☒ Có ☐ Không Consistent: ☐ Có ☒ Không

h2 ≤ h* ở mọi nút (11≤11, 7≤9, 6≤7, 4≤6, 5≤5, 1≤2) nên admissible. Nhưng không consistent: cạnh A–C có h2(A)=11 > c(A,C)+h2(C) = 4+6 = 10; cạnh E–F có h2(E)=5 > c(E,F)+h2(F) = 3+1 = 4.

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai (mọi hành động có chi phí ≥ ε > 0, mọi heuristic được dùng đều consistent).

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☒ Sai

Giải thích: Tree search không nhớ nút đã thăm nên DFS có thể đi vòng lặp vô hạn trong đồ thị có chu trình.

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☒ Đúng ☐ Sai

Giải thích: BFS duyệt theo độ sâu, tìm đích ở độ sâu hữu hạn nếu có.

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☒ Đúng ☐ Sai

Giải thích: Giới hạn độ sâu tăng dần nên chắc chắn chạm độ sâu của lời giải.

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ☒ Sai

Giải thích: Graph Search bỏ các nút đã thăm nên frontier nhỏ hơn hoặc bằng; với đồ thị không chu trình có thể bằng nhau, không "luôn lớn hơn".

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

☐ Đúng ☒ Sai

Giải thích: Graph search phải lưu tập explored nên bộ nhớ không còn nhỏ như DFS.

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

☒ Đúng ☐ Sai

Giải thích: $\min(h1(s),h2(s)) \le \min(c+h1(s'), c+h2(s')) = c + \min(h1(s'),h2(s'))$.

b) Điền heuristic admissible nhưng không consistent (S bắt đầu, G đích; S–A 1, S–C 2, A–G 3, C–G 2).

Trạng thái	**h(s)**
S	4
A	0
C	2
G	0

Chi phí thật h*: S=4, A=3, C=2, G=0. Admissible vì $4 \leq 4$, $0 \leq 3$, $2 \leq 2$. Không consistent vì cạnh S–A: $h(S)=4 > c(S,A)+h(A) = 1+0 = 1$.